

# Recurrent Neural Networks Part 1

## Agenda

Motivations

Word Embeddings

word2vec and GloVe

Sequential Data

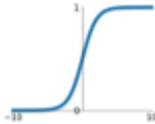
RNNs

# Recap: Image Classification

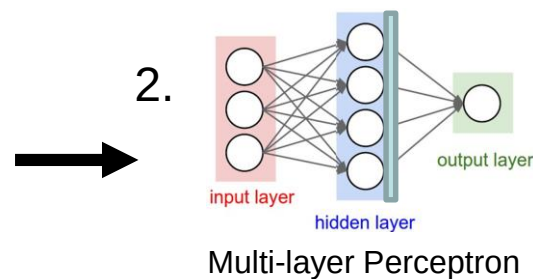
- Past weeks...
  - Classification of Spatial Data

**Input:** image



1.   $\hat{y} = \varphi \left( \sum_{i=0}^p w_i x_i = w^T x \right)$

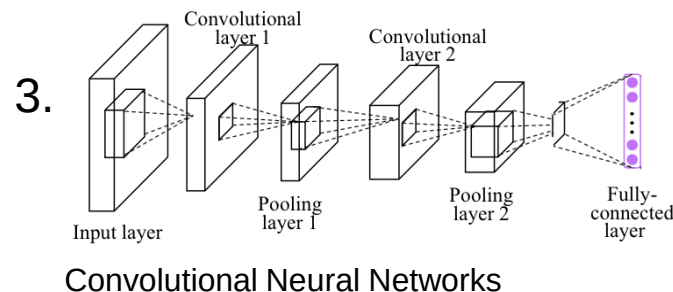
Logistic Regression



**Output:** pick the correct class

1 - car

0 - truck



# Recap: Image Generation

- Past weeks...
- Generation of Spatial Data

**Input:** provide condition (i.e., class) and sampling

1 - car

0 - truck

x

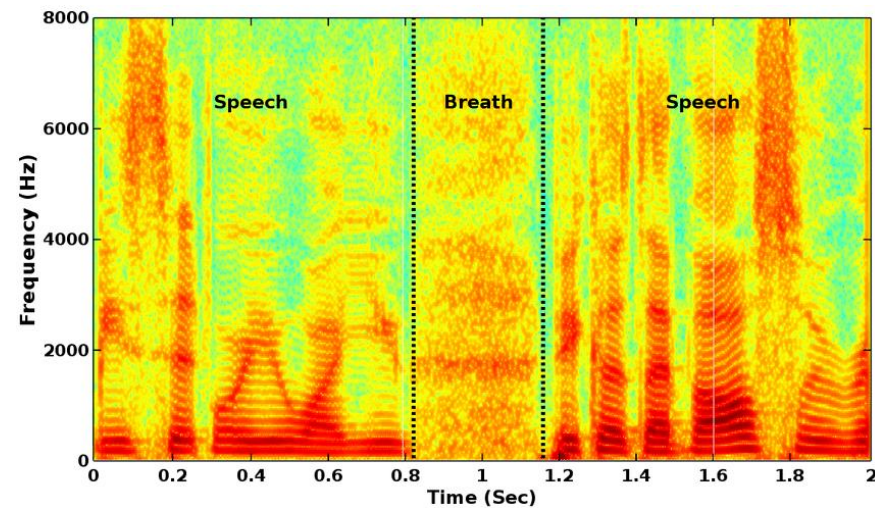
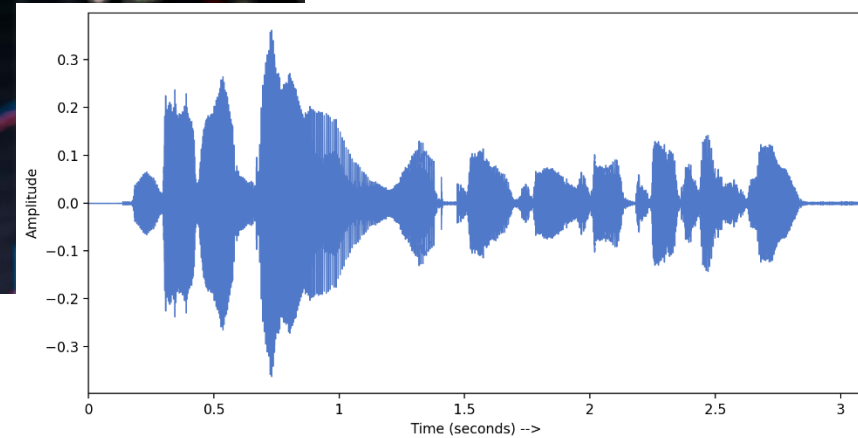
Embedding Space

**Generated Image**



Source: [TopMark Funding](#)

# Sequential Data

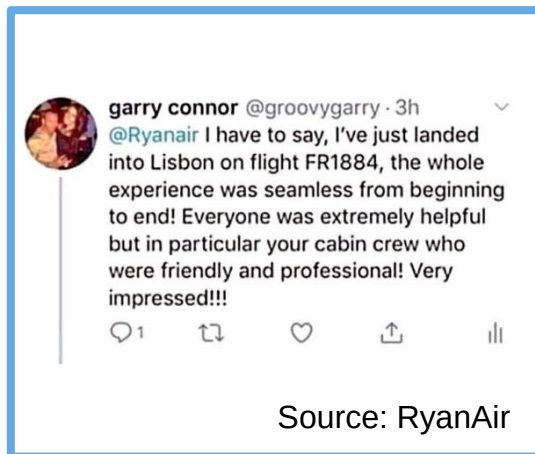


I'M GOING TO THE THEATER = ICH GEHE INS THEATER  
I'M GOING TO THE CINEMA = ICH GEHE INS KINO  
KINO

# Today: Text Classification

- Next three weeks...
  - Classification of Sequential Data

**Input: text**



**Output: pick the correct sentiment**

1 - positive

0 - negative

# Motivation -1

➤ Q: What makes working with text more difficult than working with images?

Hint: Consider the Titanic dataset from the review tutorial

| 1  | Passenger | Survived | Pclass | Name                                          | Sex    | Age | SibSp | Parch | Ticket    | Fare    | Cabin | Embarked |
|----|-----------|----------|--------|-----------------------------------------------|--------|-----|-------|-------|-----------|---------|-------|----------|
| 2  | 1         | 0        | 3      | Braund, Mr. Owen Harris                       | male   | 22  | 1     | 0     | A/5 21171 | 7.25    |       | S        |
| 3  | 2         | 1        | 1      | Cumings, Mrs. John Bradley (Florence Briggs T | female | 38  | 1     | 0     | PC 17599  | 71.2833 | C85   | C        |
| 4  | 3         | 1        | 3      | Heikkinen, Miss. Laina                        | female | 26  | 0     | 0     | STON/O2.  | 7.925   |       | S        |
| 5  | 4         | 1        | 1      | Futrelle, Mrs. Jacques Heath (Lily May Peel)  | female | 35  | 1     | 0     | 113803    | 53.1    | C123  | S        |
| 6  | 5         | 0        | 3      | Allen, Mr. William Henry                      | male   | 35  | 0     | 0     | 373450    | 8.05    |       | S        |
| 7  | 6         | 0        | 3      | Moran, Mr. James                              | male   |     | 0     | 0     | 330877    | 8.4583  |       | Q        |
| 8  | 7         | 0        | 1      | McCarthy, Mr. Timothy J                       | male   |     |       |       |           |         |       |          |
| 9  | 8         | 0        | 3      | Palsson, Master. Gosta Leonard                | male   |     |       |       |           |         |       |          |
| 10 | 9         | 1        | 3      | Johnson, Mrs. Oscar W (Elisabeth Vilhelmina E | femal  |     |       |       |           |         |       |          |
| 11 | 10        | 1        | 2      | Nasser, Mrs. Nicholas (Adele Achem)           | femal  |     |       |       |           |         |       |          |
| 12 | 11        | 1        | 3      | Sandstrom, Miss. Marguerite Rut               | femal  |     |       |       |           |         |       |          |
| 13 | 12        | 1        | 1      | Bonnell, Miss. Elizabeth                      | femal  |     |       |       |           |         |       |          |
| 14 | 13        | 0        | 3      | Saunderscock, Mr. William Henry               | male   |     |       |       |           |         |       |          |
| 15 | 14        | 0        | 3      | Andersson, Mr. Anders Johan                   | male   |     |       |       |           |         |       |          |
| 16 | 15        | 0        | 3      | Vestrom, Miss. Hulda Amanda Adolfina          | femal  |     |       |       |           |         |       |          |
| 17 | 16        | 1        | 2      | Hewlett, Mrs. (Mary D Kingcome)               | femal  |     |       |       |           |         |       |          |
| 18 | 17        | 0        | 3      | Rice, Master. Eugene                          | male   |     |       |       |           |         |       |          |
| 19 | 18        | 1        | 2      | Williams, Mr. Charles Eugene                  | male   |     |       |       |           |         |       |          |
| 20 | 19        | 0        | 3      | Vander Planke, Mrs. Julius (Emelia Maria Vanc | femal  |     |       |       |           |         |       |          |
| 21 | 20        | 1        | 3      | Masselmani, Mrs. Fatima                       | femal  |     |       |       |           |         |       |          |
| 22 | 21        | 0        | 2      | Fynney, Mr. Joseph J                          | male   |     |       |       |           |         |       |          |



Source: [Willy Stöwer](#)

# Word Inputs

- Need a way to convert words into numerical features.
- One way is to assign each word with a unique number:
  - e.g., female = 1, male = 0,
  - e.g., favourite colour: red = 1, blue = 2, green = 3, yellow = 4, etc.).
- Q: There is a problem with this... can you say that Yellow > Red? Or that Female > Male?

# One-Hot Encoding

- For **word (or categorical data)** where there is no order, integer encoding is not ideal.
  - Assuming an order may lead to poor performance.
- A better way... convert words/string into numerical features with **one-hot encoding**.
- For example:
  - We can convert three categories as follows:

|        |    |           |
|--------|----|-----------|
| Red    | -> | [1, 0, 0] |
| Green  | -> | [0, 1, 0] |
| Yellow | -> | [0, 0, 1] |

# One-Hot Encoding Limitations

- Q: What are some **disadvantages** of using one-hot encoding for embedding words?
  1. Dimensions grow with number of words
  2. One-hot encoding assumes each word is completely independent

Creating word embeddings would address both problems!

# Word Embeddings

- Q: How can we create good word embedding?  
Could we represent words by the number and type of characters?
- Meaning comes from the sequence of characters and how they are used in conjunction with other words

# Word Embeddings Con't

- A: We could try something similar to the **Autoencoder architecture** we introduced earlier in the course.

# Creating Word Embeddings

Encoder: **word(??)** -> embedding

Decoder: embedding -> **target(???)**

➤ Two things to Consider:

1. How do we encode the word?
2. What is our target?

# 1. Encoding the input word

- We can use the **one-hot encoding** where each word is represented as an “index”
    - Not great, but we must start somewhere
    - Makes no assumptions about the words
  - If there are 10,000 words, then the one-hot encoding has a length of 10,000
- ex. “happy” -> [0, 0, 0, 0, ... , 1, ... , 0, 0, 0, 0]

# Word Embeddings Con't

## **Encoder:**

one-hot embedding  $\rightarrow$  low-dim embedding

## **Decoder:**

low-dim embedding  $\rightarrow$  ???

Q: What is our target?

# Word Embeddings Con't

➤ **Key idea:** the meaning of a word depends on its context, or other words that appear nearby...

...There is evidence that children learn new words based on their surrounding words.

## 2. Specifying the Target

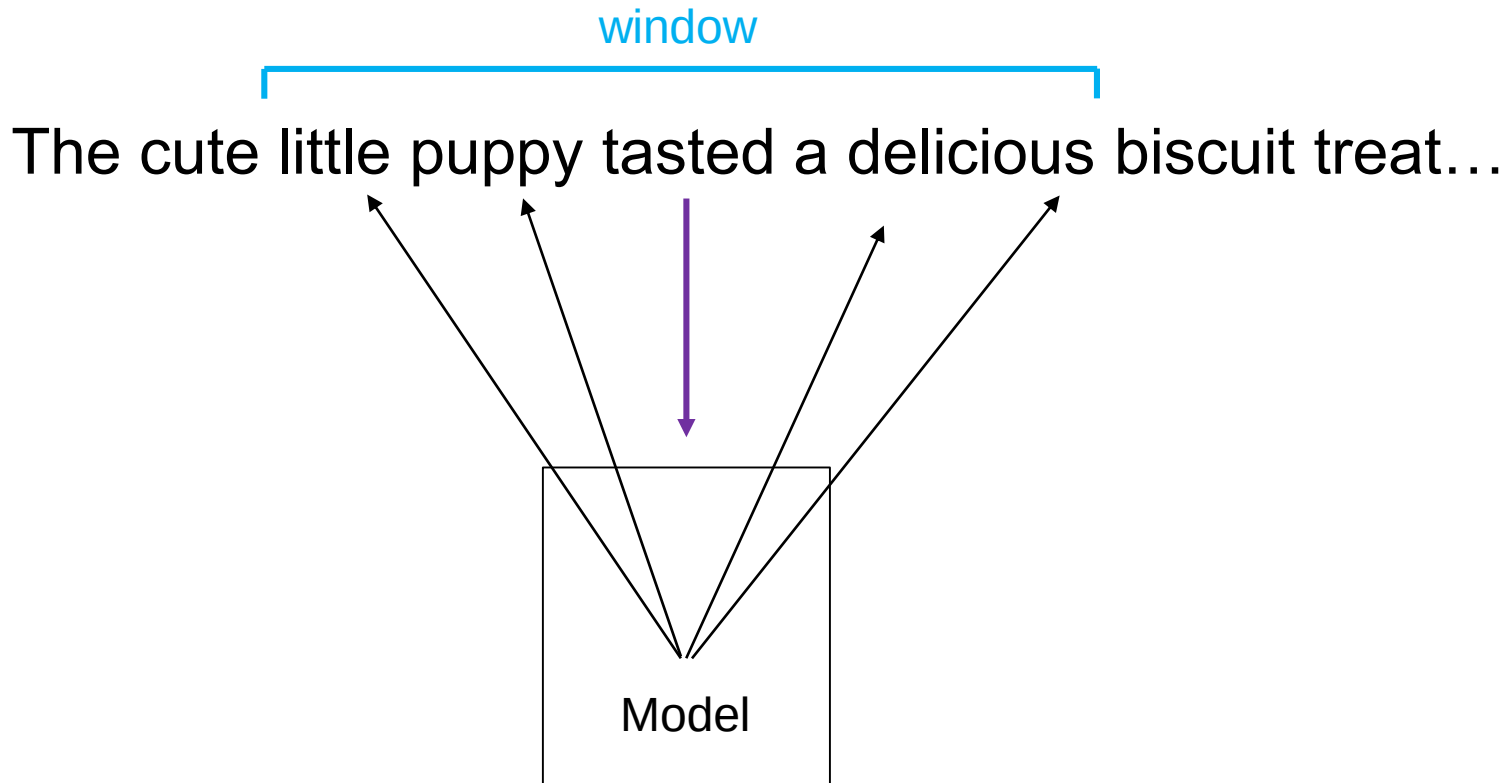
### **Encoder:**

one-hot embedding -> low-dim embedding

### **Decoder:**

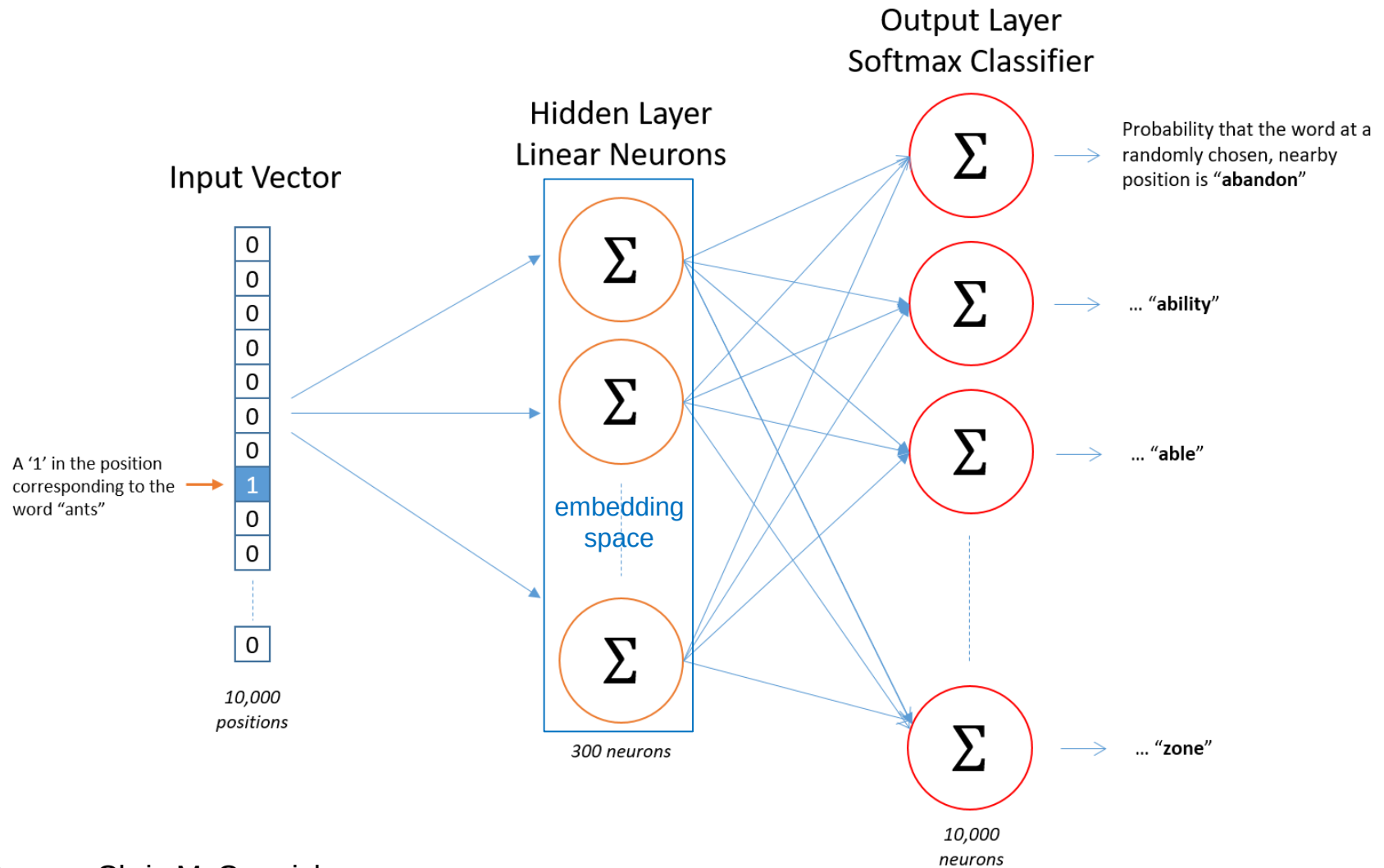
low-dim embedding -> **nearby words**

# Skip-Gram Approach

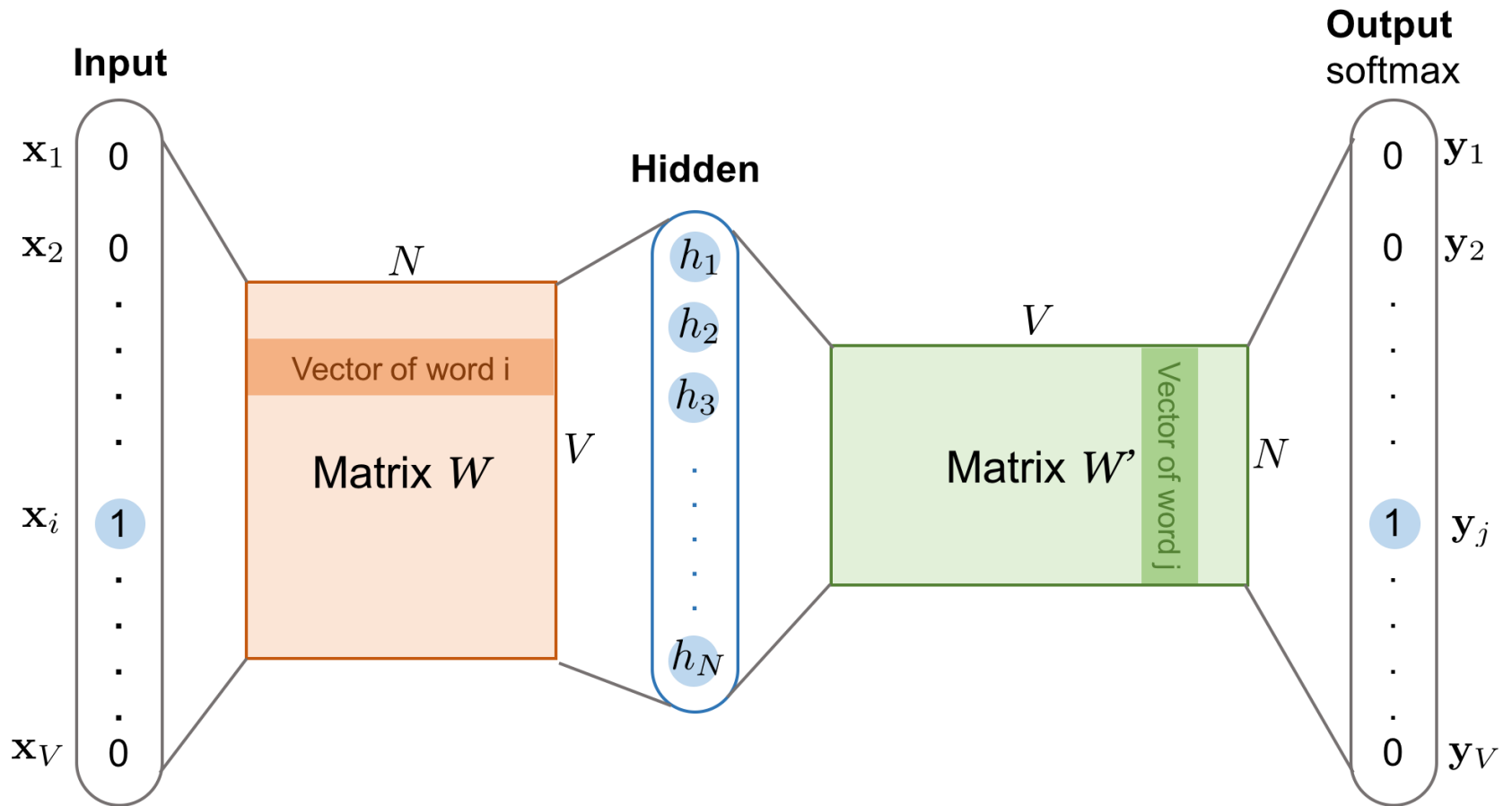


Q: What kind of learning is this?

# word2vec Architecture



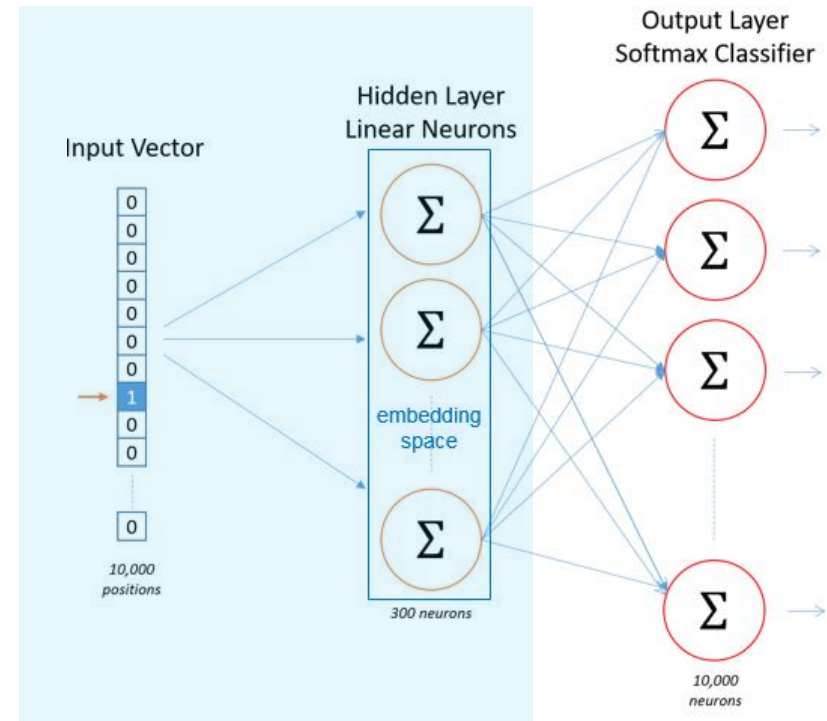
# word2vec Architecture Con't



# Word Embeddings: word2vec

- **The decoder is only used for training!**
- After the model is trained, we discard the decoder and only keep the weights from the encoder.
- Result: Words with similar context words mapped to similar embeddings

word2vec (encoder)



# Early Word Embedding Models

1. **word2vec** is an early word embedding model that is based on a neural network architecture.

Mikolov et.al., 2013

2. **GloVe** is a word embeddings model developed as an open-source project at Stanford that became widely accessible (i.e., like AlexNet).

➤ Based on a co-occurrence matrix

Pennington et.al., 2014

word2vec and GloVe produce similar results!

Let's Look at Some Code

# Example: PyTorch GloVe Embeddings

```
import torch
import torchtext
```

```
glove = torchtext.vocab.GloVe(name="6B", dim=50)
glove['cat']
```

```
tensor([ 0.4769, -0.0846, ... ])
```

Torchtext is currently deprecated, need to pip install for code to work!

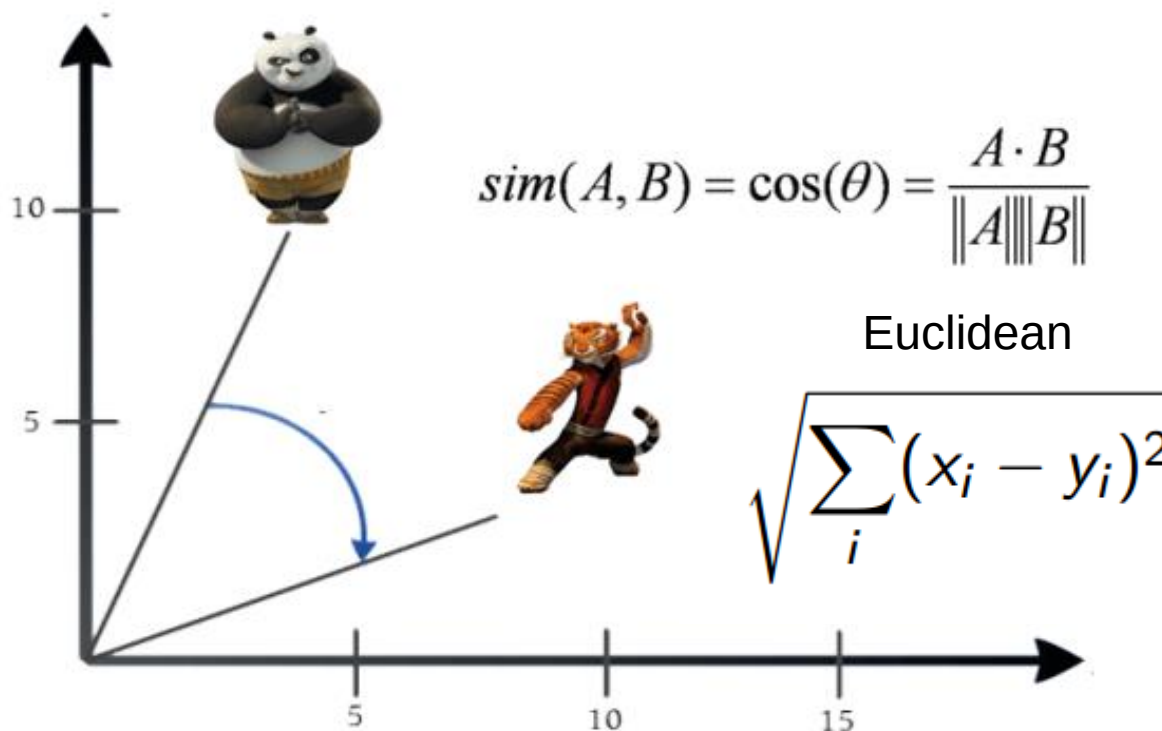
- Use `torchtext` package to load pre-trained GloVe embeddings
- First time you run it will load an 862MB file containing pretrained embeddings
- “6B” was trained on Wikipedia 2014 corpus
- `dim` is embedding size

# Distance Measures

- In order to talk about which words have “similar” GloVe embeddings, We need to introduce a measure of distance in the embedding space.
- Two popular approaches:
  1. Euclidean Distance
  2. Cosine Similarity

# Euclidean Distance vs Cosine Similarity

- **Euclidean Distance:** 2-norm difference between two vectors
- **Cosine Similarity:** angle between the two vectors



Cosine similarity is useful when we want a distance measure that is **invariant to the magnitude of the vectors**.

# Computing Distances in PyTorch

## ➤ Euclidean Distance:

```
torch.norm(glove['cat'] - glove['dog'])
```

## ➤ Cosine Similarity:

```
torch.cosine_similarity(  
    glove['cat'].unsqueeze(0),  
    glove['dog'].unsqueeze(0))
```

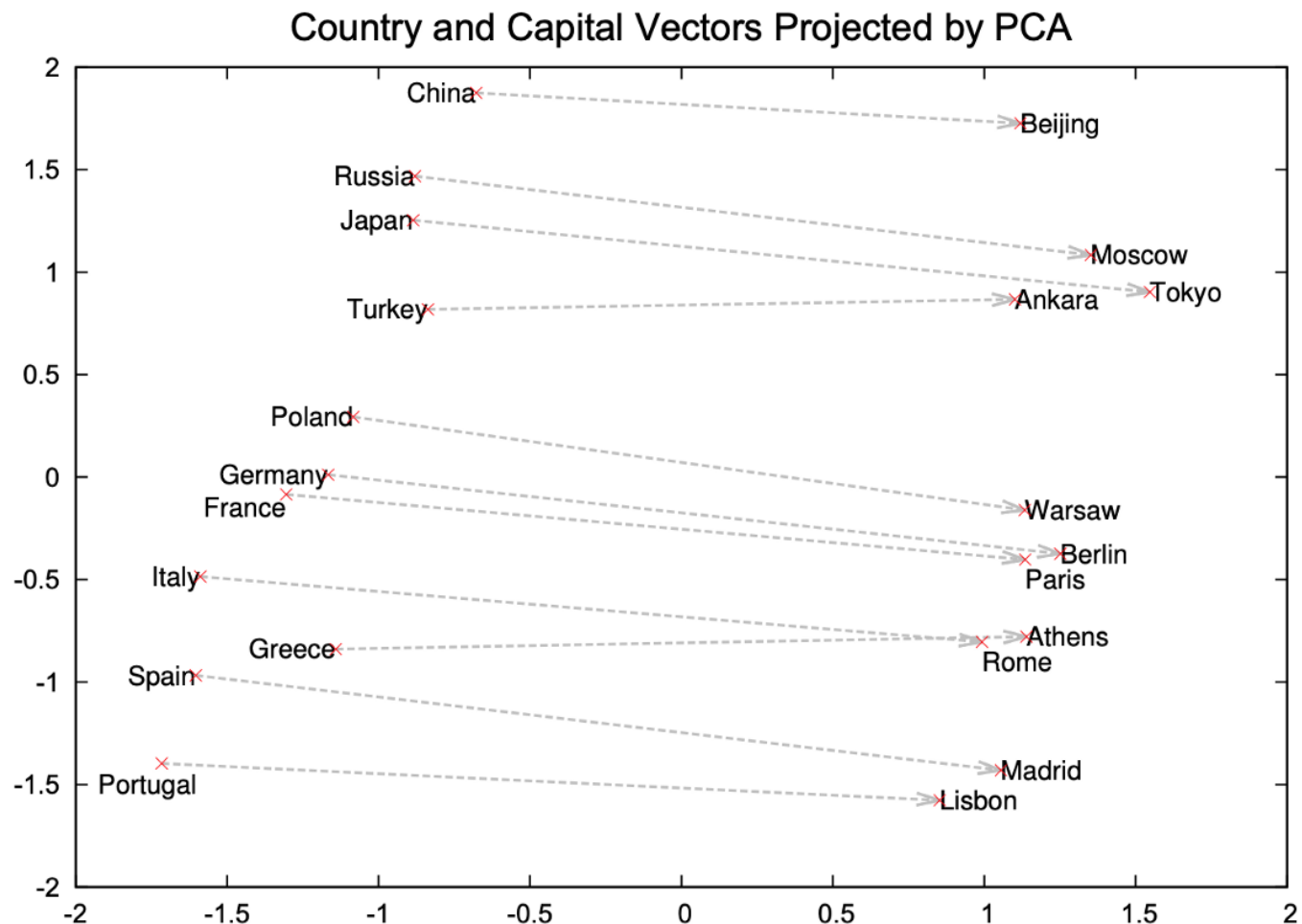
## ➤ Let's look at similarities between word embeddings in PyTorch.

# Word Analogies

- One surprising thing about the embedding space is the extent of its structure.
- We often see relationships like this in GloVe embeddings:

$$\textit{king} - \textit{man} + \textit{woman} \approx \textit{queen}$$

# Word Analogies



Source: [Mikolov et al., 2013](#)

# Bias in Word Embeddings

- These word analogies show that machine learning models are not unbiased.

*doctor - man + woman ≈ ??*

- Machine learning models learn the biases present in the data it is trained on.

# Sentiment Analysis

# Motivation -2

➤ Q: How do we deal with sequential data?

Feedback/Review

Response



**garry conner** @groovygarry · 3h  
@Ryanair I have to say, I've just landed into Lisbon on flight FR1884, the whole experience was seamless from beginning to end! Everyone was extremely helpful but in particular your cabin crew who were friendly and professional! Very impressed!!!



**Ryanair** @Ryanair

Replying to @groovygarry

Hi Garry, sorry for the inconveniences.  
Please, submit a complaint here:  
[contactform.ryanair.com](https://contactform.ryanair.com)

Ani

# Requirement: Language Modelling

## ➤ **Text Understanding**

- Question Answering

- Sentiment Analysis

## ➤ **Text Generation**

- Sentence completion

- Generating captions, sentences, stories. . .

- Translation

- . . . and more!

# Sentiment Analysis

- **Goal:** Given a piece of text, identify the sentiment that the text conveys.
- Can be applied to:
  - movie reviews
  - feedback
  - emails
  - tweets
- ❑ Difficult problem in general
- ❑ Hard to collect clean, labelled data

# Challenges with Text

- Q: How is working with text different (more challenging) from working with images?
  - Grammar, spelling
  - Many words to learn
  - Choice of working with words vs characters
  - Arbitrary length input / output
  - Different languages

# Use Public Datasets

## Sentiment 140 Data:

- 160,000 tweets
- Sentiment determined by emoticon
- Collected by students doing a course project
- Q: What are the advantages of using tweets as training data?
- Q: What are the challenges of using tweets as training data?

# Model Algorithm: Encoding Text

For each tweet in the training data:

1. Split the tweet into words
2. Look up the GloVe embedding for each word, ignoring words that don't have embeddings
3. Add up the word embeddings to obtain an **embedding for the entire tweet**
4. The tweet embedding will be the input to a neural network

# 1. Splitting into Words

*“I have to say, I’ve just landed into Lisbon on flight FR1884, the whole experience was seamless from the beginning to end! Everyone was extremely helpful but in particular your cabin crew who were friendly and professional! Very impressed!!!”*

Splitting into Words:

- i
- have
- to
- . . .
- . . .

## 2. Look up GloVe Embedding

*“I have to say, I’ve just landed into Lisbon on flight FR1884, the whole experience was seamless from the beginning to end! Everyone was extremely helpful but in particular your cabin crew who were friendly and professional! Very impressed!!!”*

Look up GloVe Embeddings:

- `i: = tensor([ 1.1891e-01, 1.5255e-01, ...])`
- `have: = tensor([ 0.9491, -0.3497, 0.4812, ...])`
- `to: = tensor([ 0.6805, -0.0393, 0.3019, ...])`
- `...`
- `...`

### 3. Add up Embeddings

*“I have to say, I’ve just landed into Lisbon on flight FR1884, the whole experience was seamless from the beginning to end! Everyone was extremely helpful but in particular your cabin crew who were friendly and professional! Very impressed!!!”*

Add up embedding:

- $i: = [1.1891e-01, 1.5255e-01, \dots]$
- $have: = [0.9491, -0.3497, 0.4812, \dots]$
- $to: = [0.6805, -0.0393, 0.3019, \dots]$
- $\dots$
- **Tweet Embedding:**  $= [16.183, 2.2113, \dots]$

# Training a Neural Network

- We should **pre-compute tweet embeddings** of all our training, validation, and test data (like some of you did in Assignment 2 transfer learning)
- Each tweet is represented by an embedding vector, which we put into a DataLoader
- Our classifier neural network will be fully-connected

# Considerations

- Q: What are the advantages / disadvantages of this architecture?
- Q: What are the advantages / disadvantages of using this dataset?
- Q: What are ethical considerations that arise from using GloVe vectors?

# Code

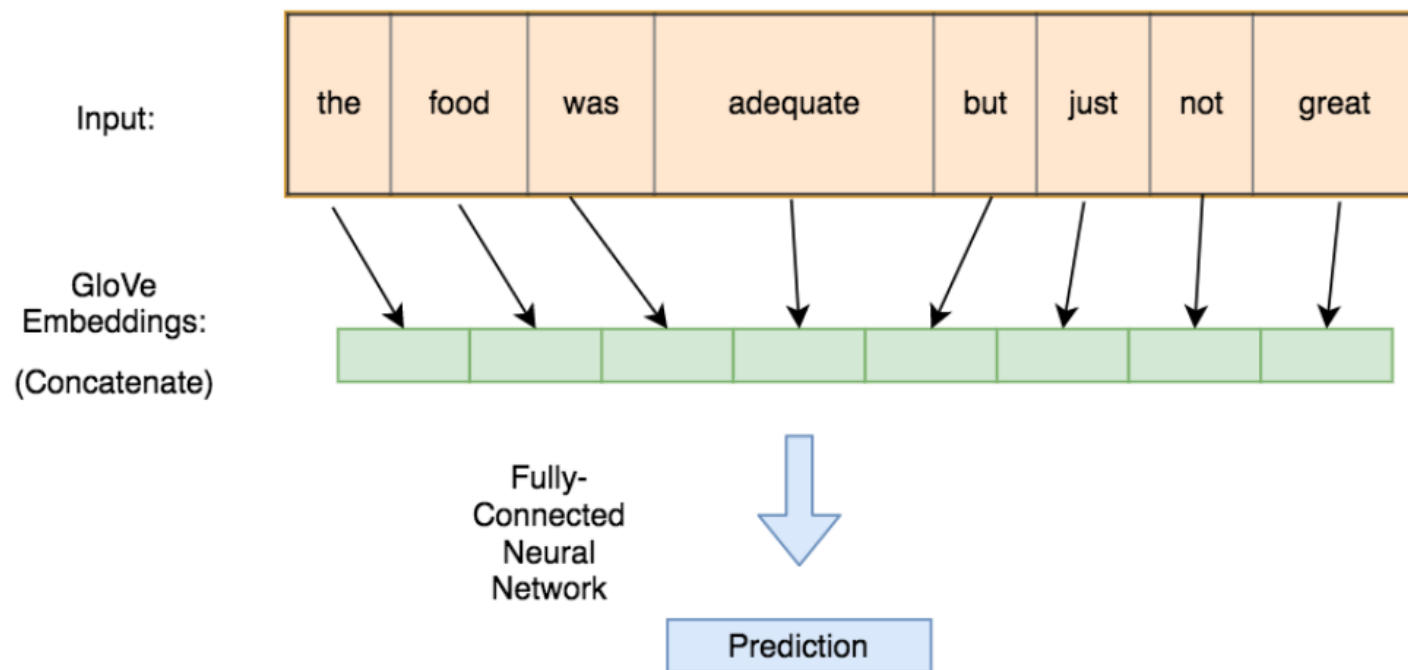
➤ Let's look at some code!

# Limitations

- These two sentences will have the same embedding in our model:
  - The food was adequate, but just not great (-)
  - The food was not just adequate, but great (+)
- . . . but they have drastically different meanings.
- Our model does not take into account the order of words
- Q: How can we improve our model?

# Idea #1

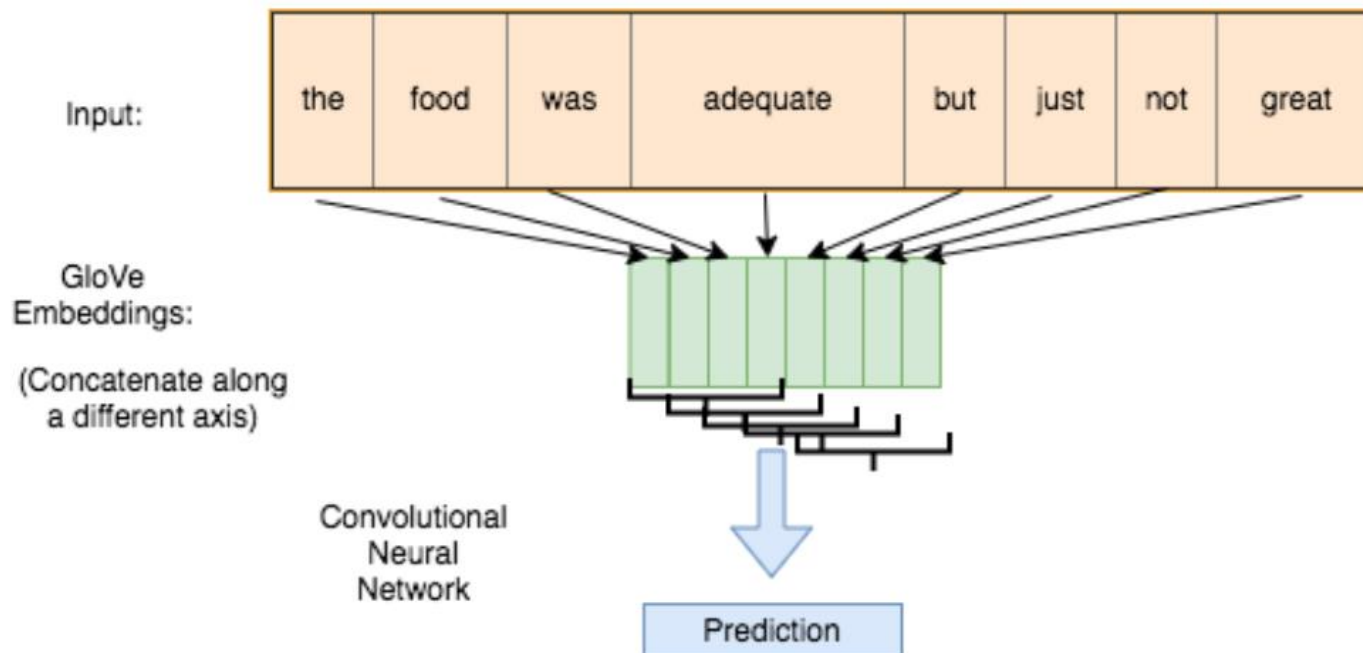
- Concatenate (and flatten) the word embeddings, then train a neural network that takes the concatenated embedding as input.



Q: What is a drawback of this idea?

# Idea #2

- Concatenate the word embeddings, then train a 1-dimensional convolutional neural network that takes the concatenated embedding as input.



Q: What is a drawback of this idea?

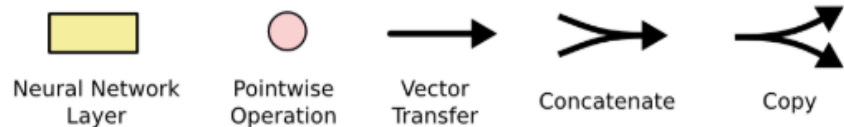
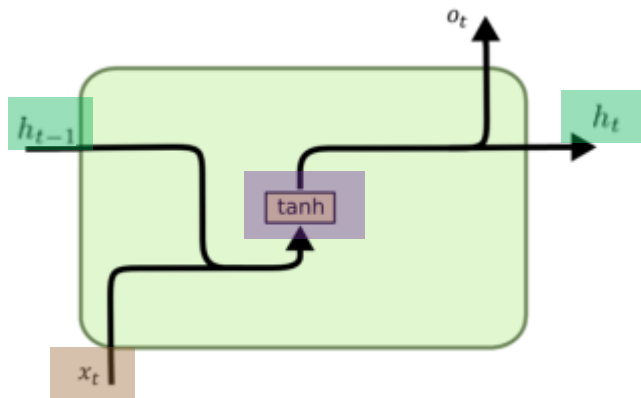
# Recurrent Neural Networks

- Want an architecture that:
  - Can take in variable-sized **sequential** input
  - Can remember things over time, or has some sort of **memory** or **state**
- Recurrent Neural Networks!

# Recurrent Neural Networks (Sequential Data)

# RNN Architecture

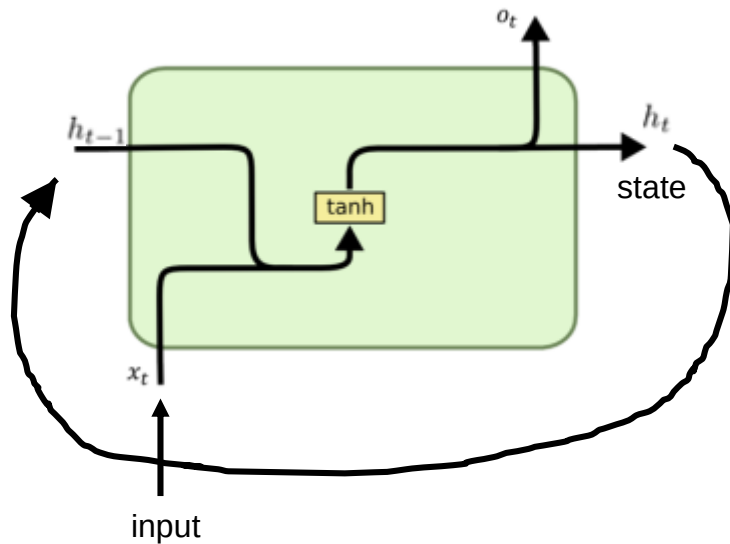
- The forward path in RNNs shares similarities with the a fully-connected neural network
  - weights applied on input and hidden state
  - tanh activation function



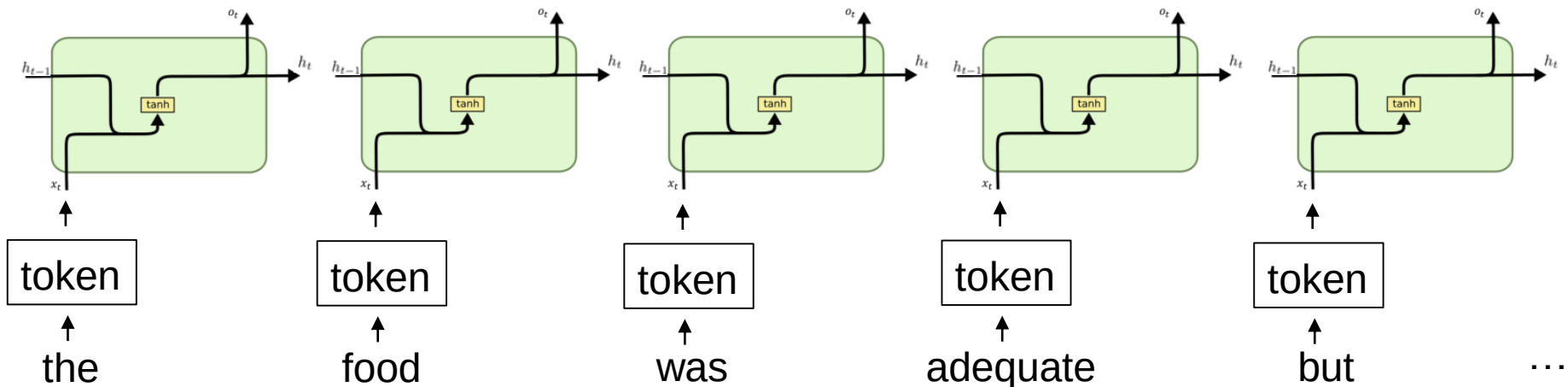
Source: Colah's blog

$$\text{hidden}(t) = \tanh(\text{weight\_hidden} * \text{hidden}(t-1) + \text{weight\_input} * (\text{input}(t)))$$

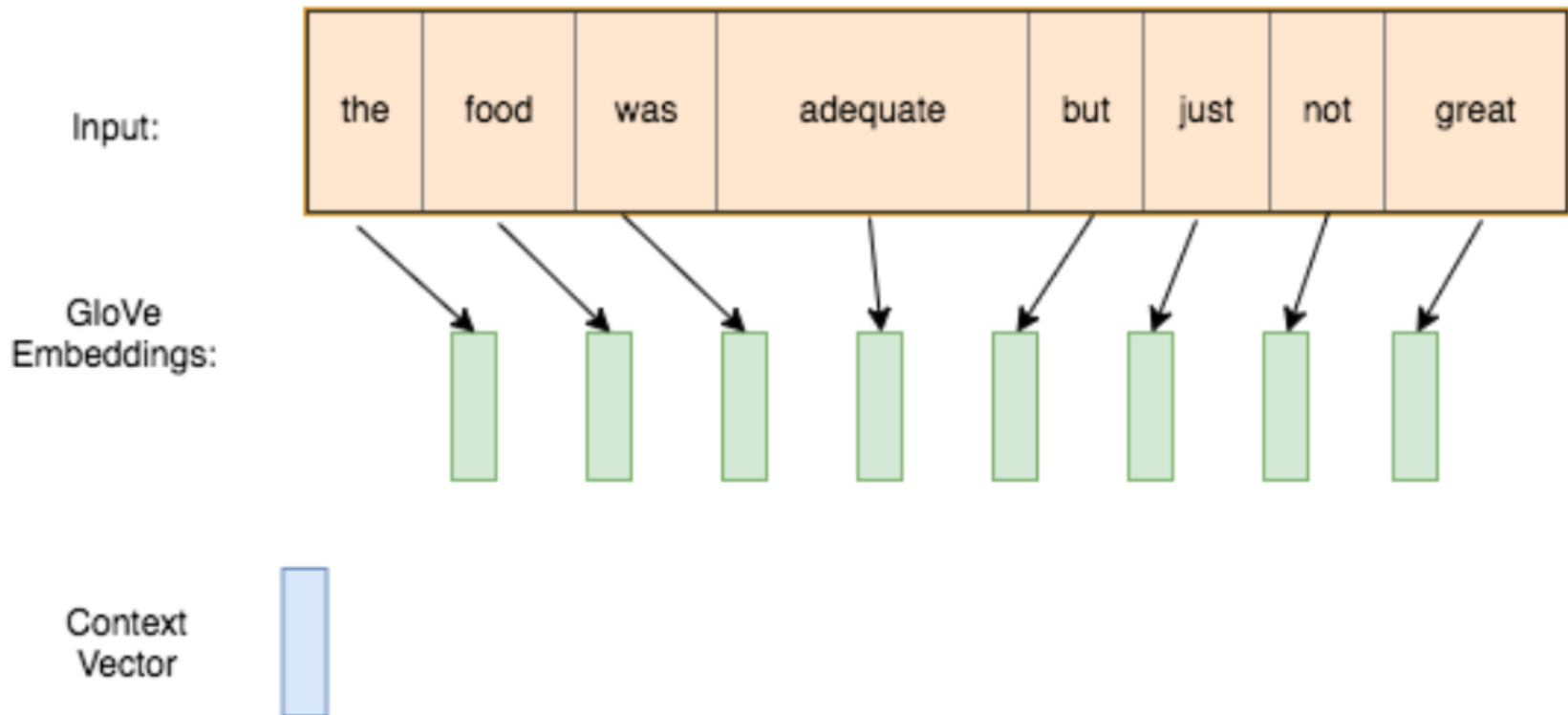
# RNN Unfolded



- Reuse the RNN module for entire length sequence
- Keeping track of the context in the hidden state ( $h$ )

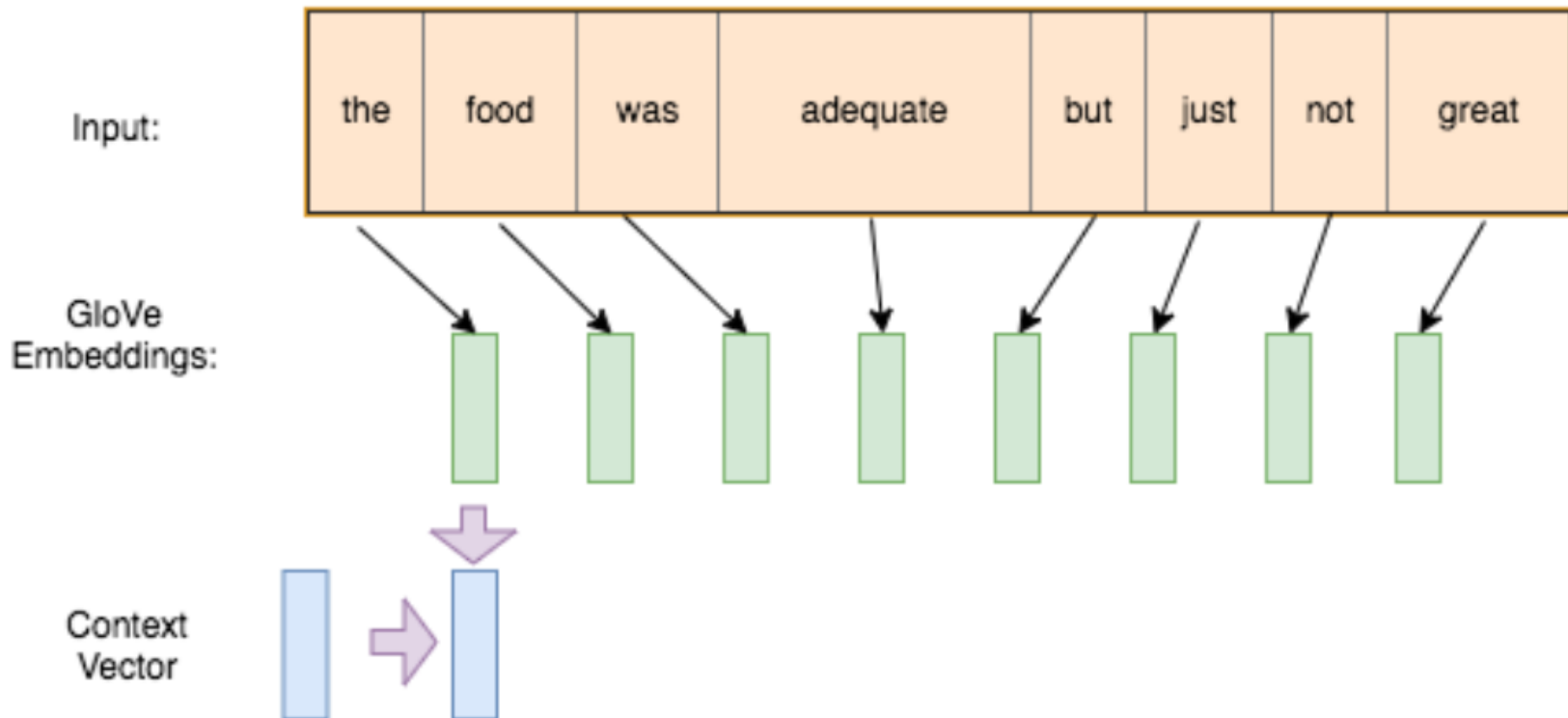


# RNN: Initial Hidden State



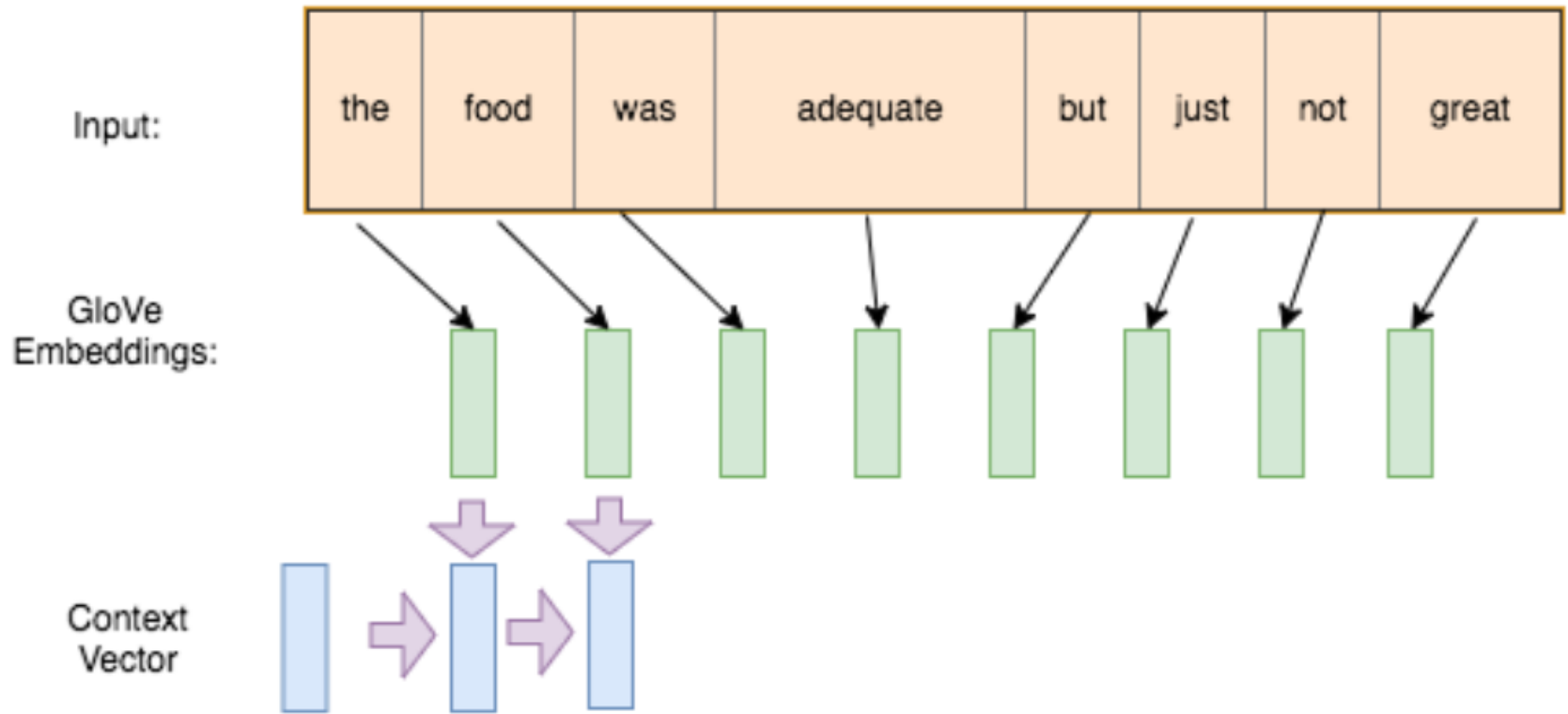
- Start with an initial **hidden state** with a blank slate (can be a vector of all zeros)

# RNN: Update Hidden State



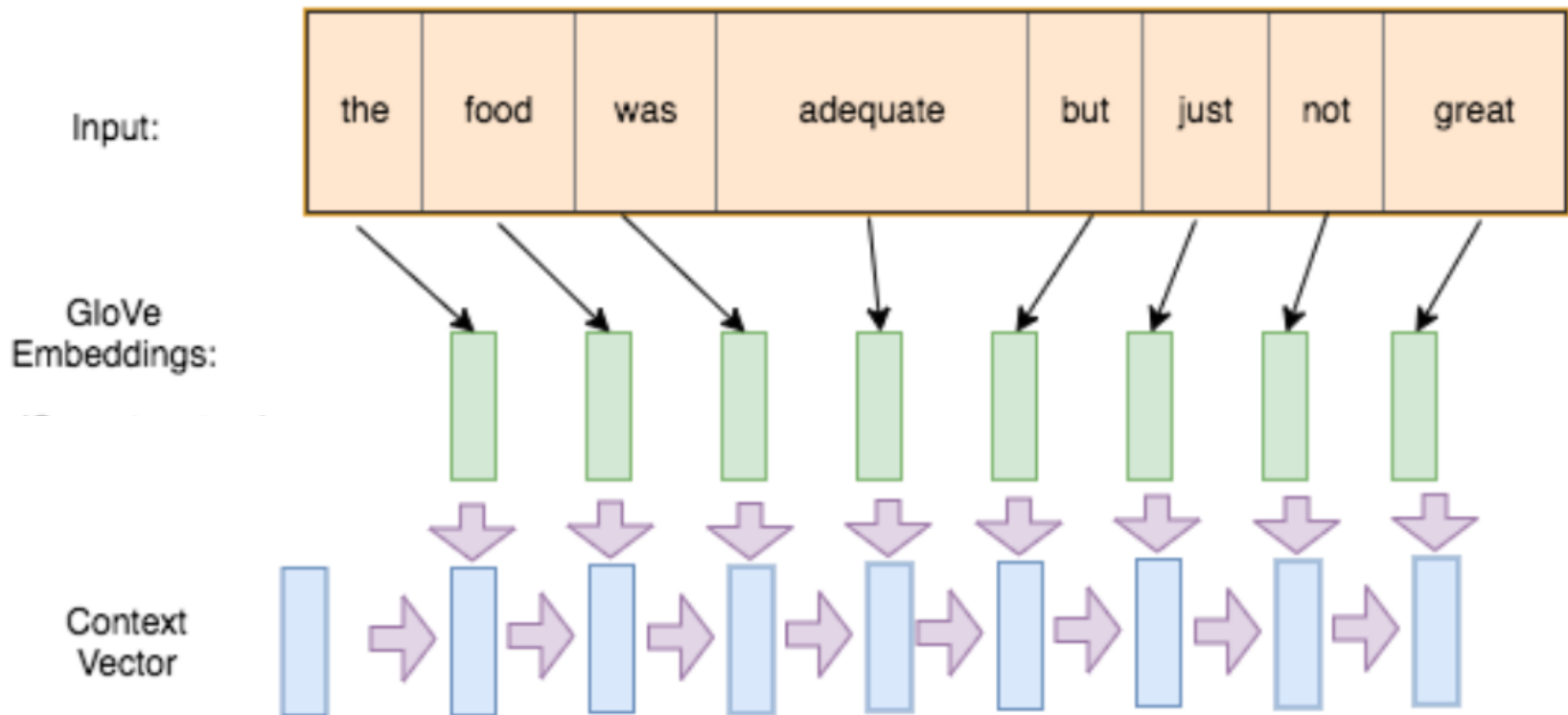
- Hidden state is updated based on previous hidden state, and the input
- `hidden = update_function(hidden, input)`

# RNN: Continue Updating Hidden State



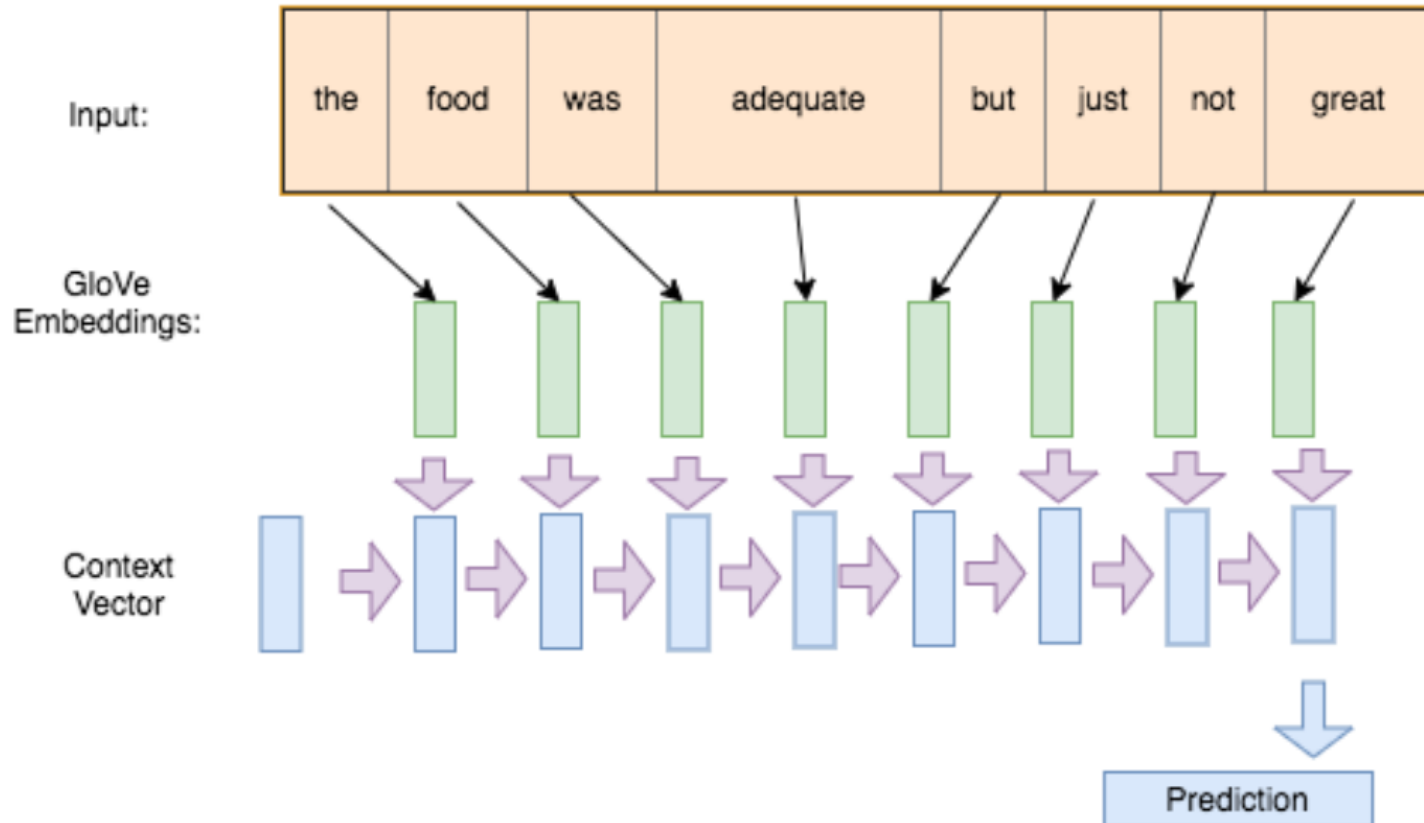
- Hidden state is updated based on previous hidden state, and the input **using the same neural network as before** (weight sharing).
- `hidden = update_function(hidden, input)`

# RNN: Last Hidden State



- Continue updating the hidden state until we run out of tokens.
- `hidden = update_function(hidden, input)`

# RNN: Compute Prediction



- Use the last hidden state as input to a prediction network
- `output = prediction_function(hidden)`

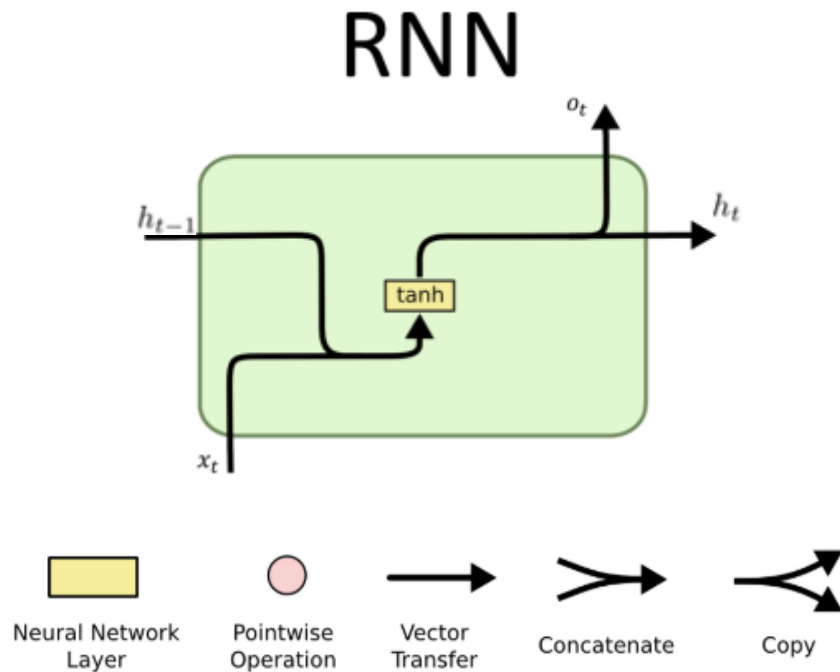
# PyTorch: RNN Layer

```
rnn_layer = nn.RNN(input_size=50,  
                    hidden_size=50,  
                    batch_first=True)
```

# dimension of the input representation

# dimension of the hidden units

# input format [batch\_size, seq\_len, repr\_dim]



# PyTorch: RNN Architecture

```
class TweetRNN(nn.Module):
    def __init__(self, input_size, hidden_size, num_classes):
        super(TweetRNN, self).__init__()
        self.emb = nn.Embedding.from_pretrained(glove.vectors)
        self.hidden_size = hidden_size
        self.rnn = nn.RNN(input_size, hidden_size, batch_first=True)
        self.fc = nn.Linear(hidden_size, num_classes)

    def forward(self, x):
        # Look up the embedding
        x = self.emb(x)
        # Set an initial hidden state
        h0 = torch.zeros(1, x.size(0), self.hidden_size)
        # Forward propagate the RNN
        out, _ = self.rnn(x, h0)
        # Pass the output of the last time step to the classifier
        out = self.fc(out[:, -1, :])
        return out

model = TweetRNN(50, 50, 2)
```

# PyTorch: RNN Training

```
def train_rnn_network(model, train, valid, num_epochs=5,
                      learning_rate=1e-5):
    criterion = nn.CrossEntropyLoss()
    optimizer = torch.optim.Adam(model.parameters(),
                                  lr=learning_rate)

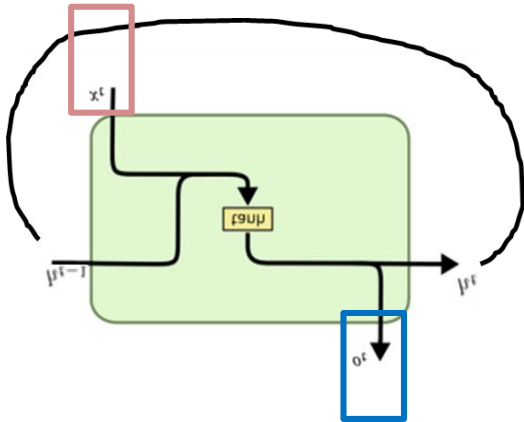
    for epoch in range(num_epochs):
        for tweets, labels in train:
            optimizer.zero_grad()
            pred = model(tweets)
            loss = criterion(pred, labels)
            loss.backward()
            optimizer.step()
```

- Training is very similar to what we've seen with other neural network architectures

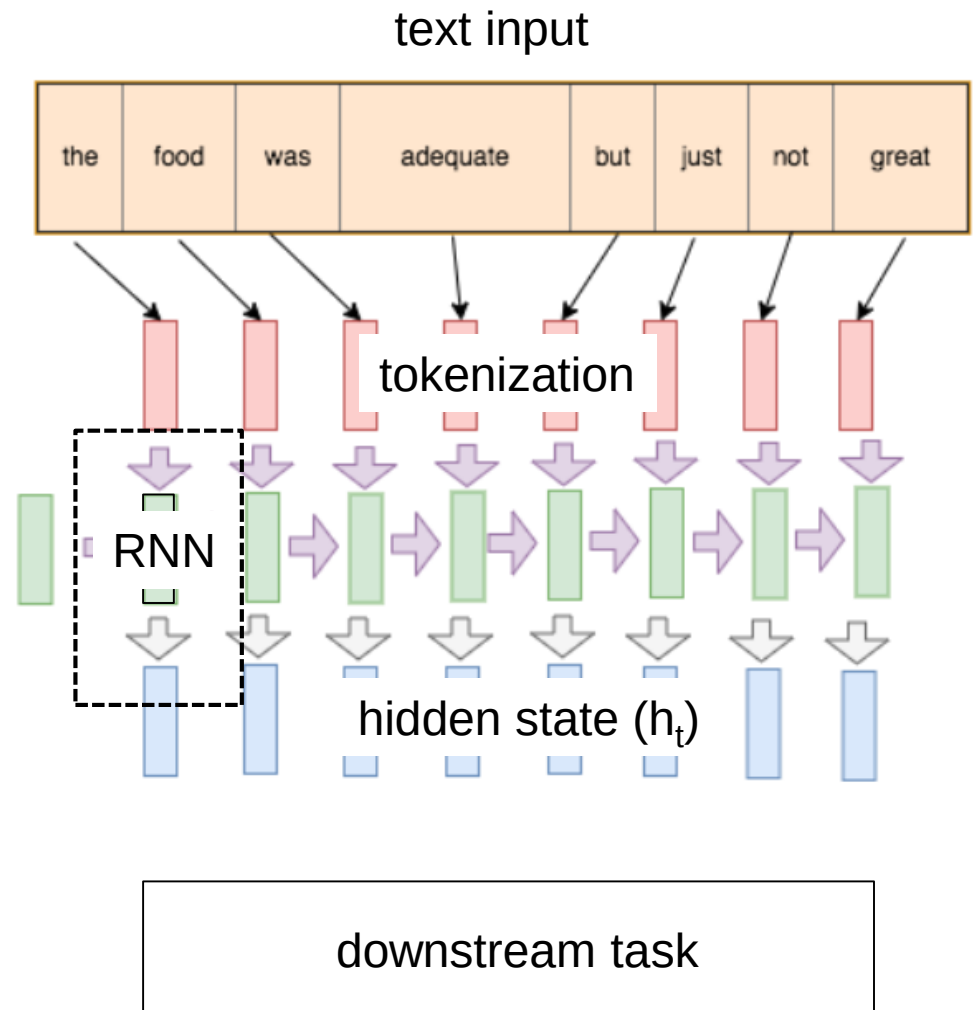
# RNN Notes

- Not all Recurrent Neural Networks use GloVe embeddings to represent the input tokens
- We could have used a one-hot encoding
  - Not practical with this example
  - Possible with a small vocabulary size
- As an example, we will see character-level tokens in Assignment 4.

# Summary: RNN Model



- Reuse the RNN module for entire length sequence
- Keeping track of the context in the hidden state ( $h$ )



# Summary: RNNs Part 1

- Processing text into **numerical values**
- Capture **meaningful representations** of words
- Simplify **downstream tasks**
  - sentiment analysis,
  - text summarizations,

## Limitation of GloVe and Word2Vec

- word-level embedding, fixed meaning independent of context,
- **Next time: sentence-level embedding (or greater)**

# Week 8 - Announcements

# Assignment 4 – Sequential Data

- The final assignment
- Due Tue, Nov 18th at 11pm
- Part 1: Train an RNN to identify sentiment in movie reviews
- Part 2: Use a pretrained transformer model to identify sentiment in movie reviews
- Bonus: Explore generative LSTM to build a small language model to improve movie reviews. This is like the transfer learning seen before.

# Project Deliverables

Details on project deliverables have been posted on Quercus:

1. Project Progress due Nov 11th at 11pm
2. Video Presentations due Nov 30th at 11pm
3. Final Project Report due Dec 5th at 11pm